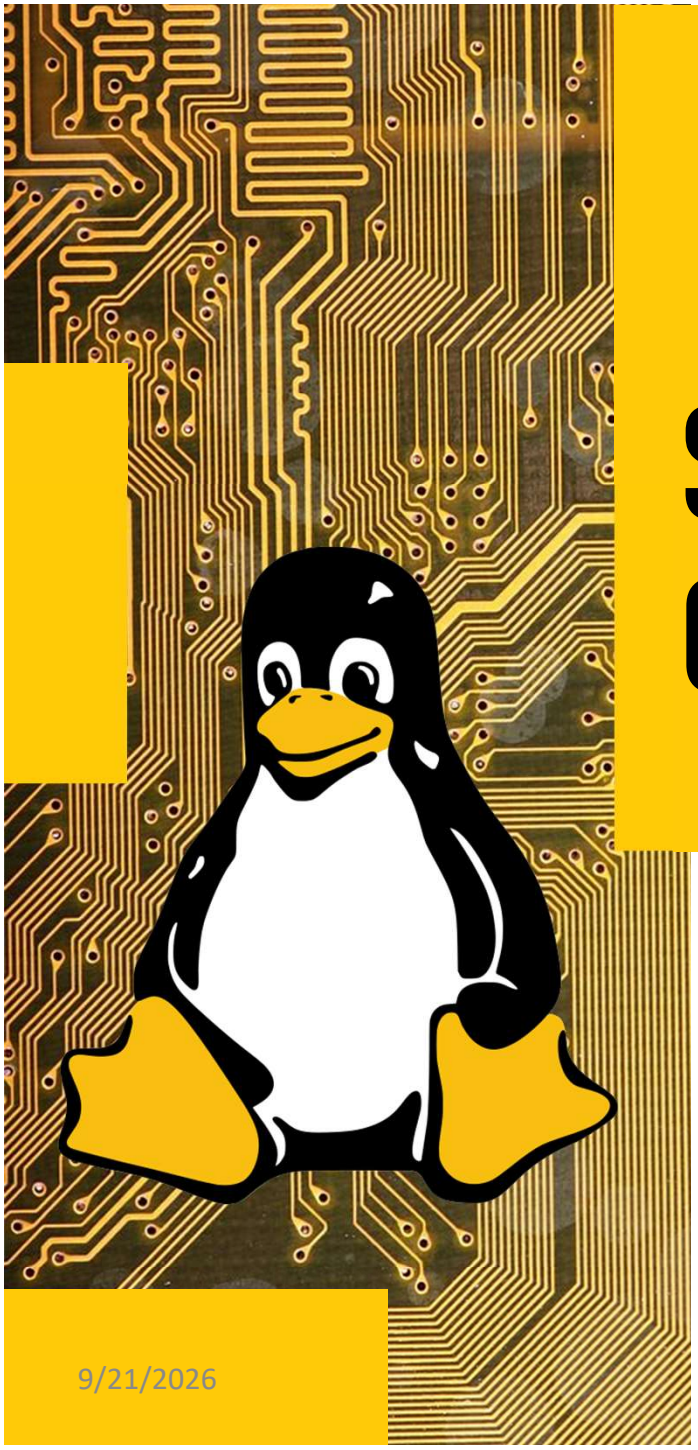


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Inter-Process Communication Pipes and dup2

INSTRUCTOR: Jalolliddin Yusupov

WEEK III Course Topics & Weekly Lecture Breakdown

Lecture Topic:
Inter-Process Communication - Pipes and FIFOs (TLPI Ch 44)

- Why processes cannot see each other's memory; message passing and shared memory. Pipes: creation, inheritance across fork, closing unused ends, blocking and end-of-file rules, atomic writes, the SIGPIPE signal; redirection with dup2. Named pipes (FIFOs) and a simple client-server built on them; the multi-stage pipeline.

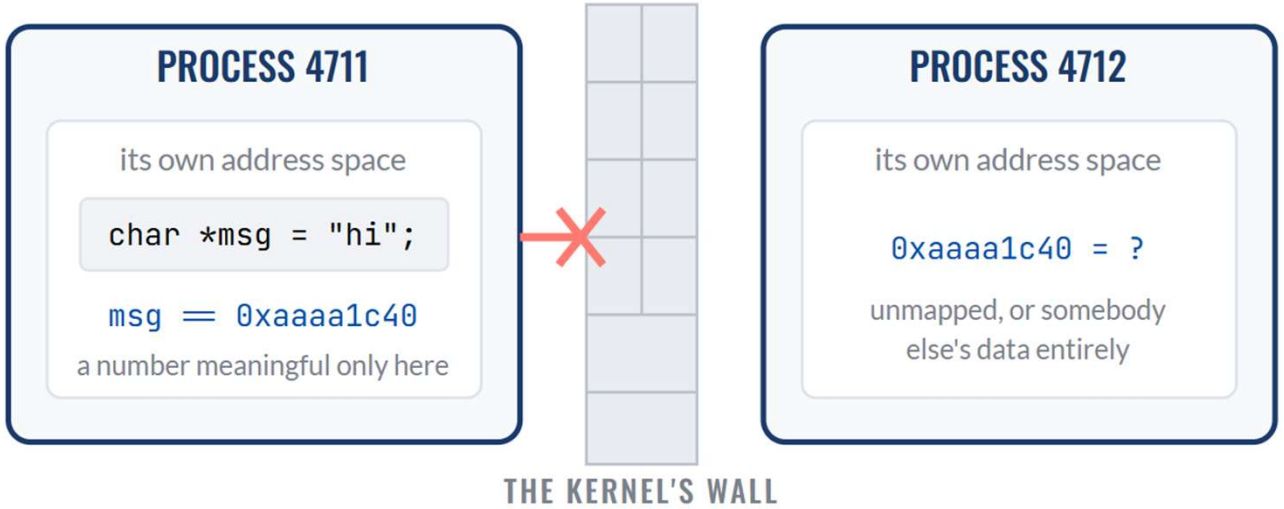
The promise on the syllabus

Week III, from the course-organisation deck: *"why processes cannot see each other's memory; pipes: creation, inheritance across* `fork`*, closing unused ends, blocking and end-of-file rules, atomic writes, the* `SIGPIPE` *signal; redirection with* `dup2`*; named pipes (FIFOs) and a simple client–server; the multi-stage pipeline."* All of it is here.

Reading

TLPI ch. 44 (*The Linux Programming Interface*, the course's system-programming reference). Silberschatz §3.4–3.7. Manual pages: `pipe(2)`, `pipe(7)`, `fifo(7)`, `dup2(2)`, `mkfifo(3)`.

The Problem: Two Private Address Spaces



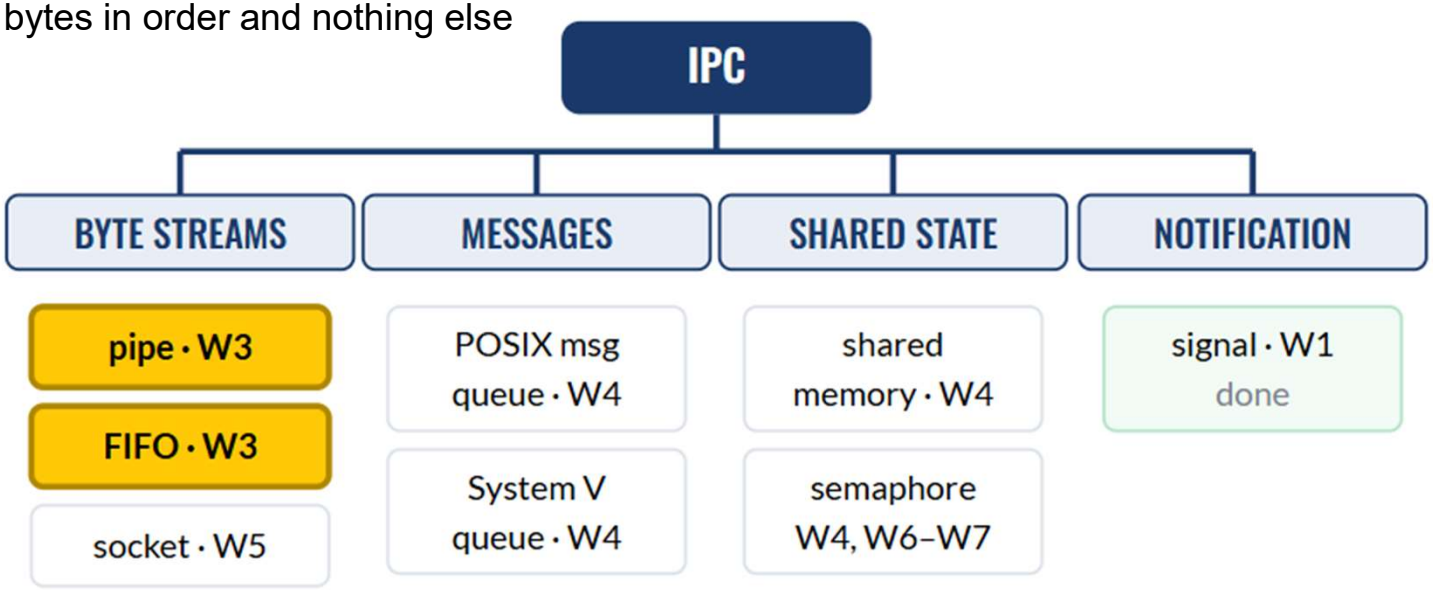
Why the wall cannot simply be lowered

Each process gets its own **virtual address space**, and the hardware translates every pointer through a map that belongs to *that process alone* (the machinery is **W12–W13**). Two processes can hold the identical number 0xaaaa1c40 and reach two unrelated bytes. Handing a pointer across is not a bug you can patch: there is nothing on the other side to point at.

A **byte stream** carries bytes in order and nothing else

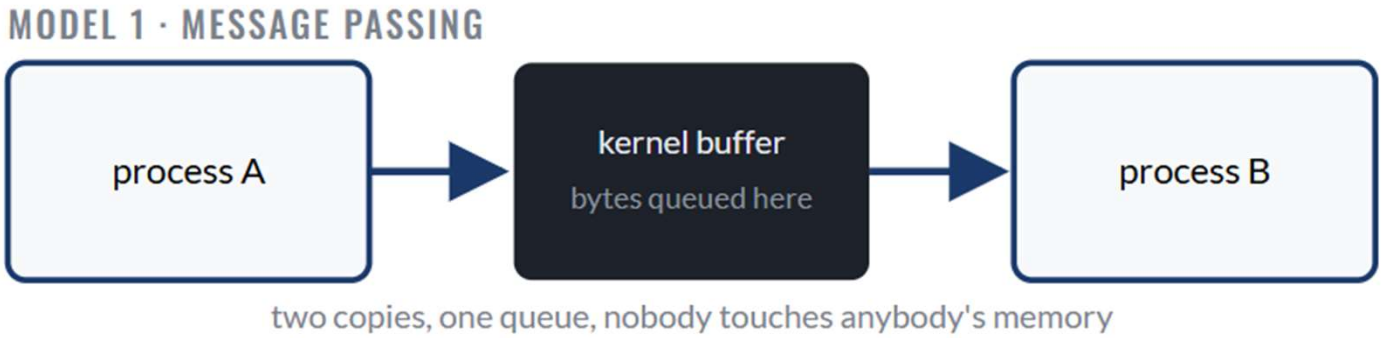
One name, in case it is new

IPC — inter-process communication: any kernel-mediated way for two processes with separate address spaces to exchange data.



Message passing— and Why Not Just Use a File

An operating system's first job is to keep processes apart, and that immediately creates its second job: **giving them a safe way to cooperate anyway.**

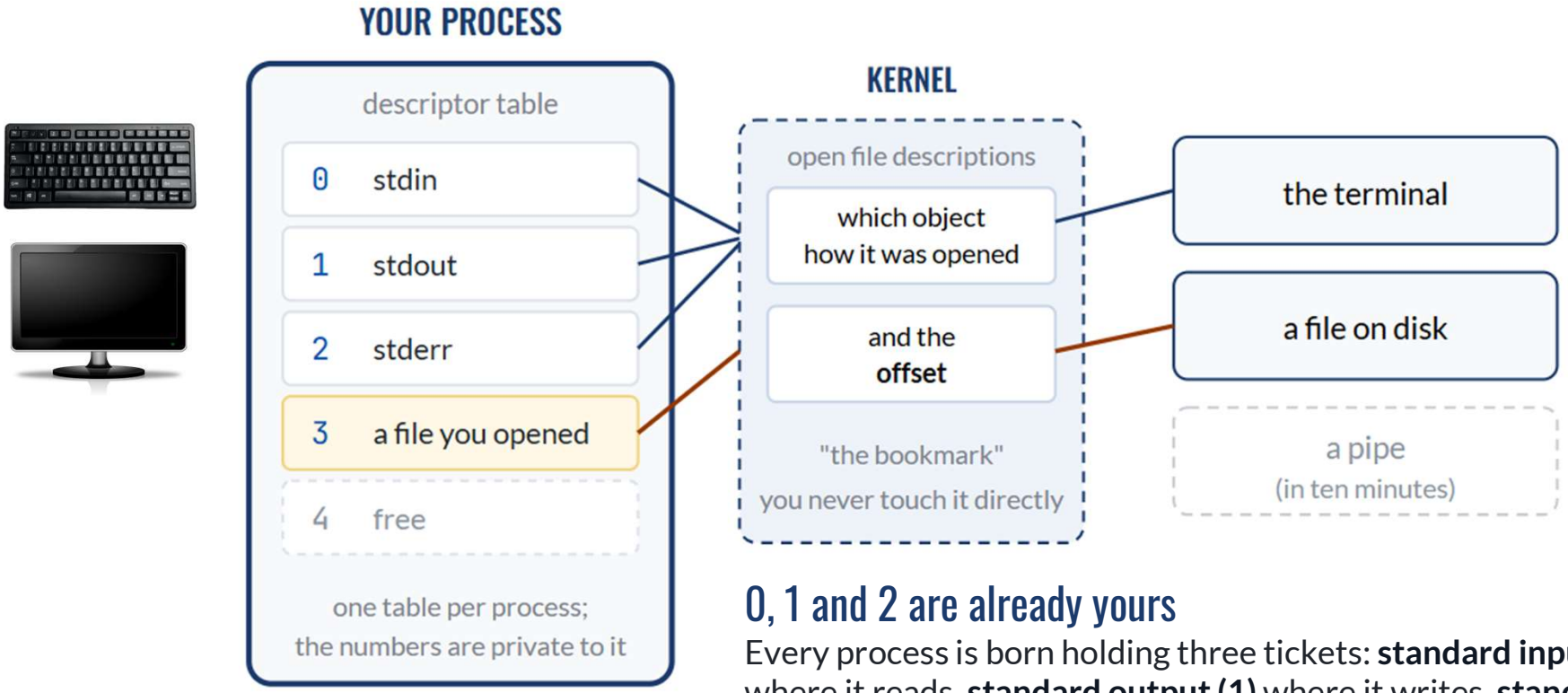


Why not a shared file?

You can, and it is wrong on four counts:

1. **No notification.** Nothing wakes you when data lands; you must keep re-reading and asking.
2. **No boundaries.** A file is bytes; where one message ends is your problem.
3. **Racing offsets.** Two writers with two independent file positions overwrite each other — and two writers who *share* one position do something stranger still.

A File Descriptor Is a Ticket Number



0, 1 and 2 are already yours

Every process is born holding three tickets: **standard input (0)** where it reads, **standard output (1)** where it writes, **standard error (2)** where it complains.

Three names, all new today

File descriptor — a small whole number your process uses as a ticket for something it has open. Each process has its own file descriptor table managed by the **OS**.
Descriptor table — your process's private list of tickets, indexed 0, 1, 2, 3 … Every process has its own.
Open file description — the kernel's bookmark behind a ticket: *which* object, *how* you opened it, and *where in it you are*.

`write()` and `read()`: the Two Calls This Week Runs On

TWO SYSTEM CALLS, TWO COUNTS

```c
/* hand n bytes to whatever ticket 1 points at, NOW */
ssize_t w = write(1, "hi\n", 3);   /* w == 3 */

/* wait for up to 64 bytes from ticket 0 */
char buf[64];
ssize_t r = read(0, buf, sizeof buf);
/* r > 0  : that many bytes arrived          */
/* r == 0 : END OF FILE - no more will come  */
/* r == -1: an error; look at errno          */
```

Two names, in case they are new

`ssize_t` — a signed size: a count, or `-1` for failure.

`errno` — C's per-call error variable: when a call returns `-1`, `errno` says why.

End of file (EOF) — `read` returning **0**, meaning *no more data will ever arrive*. At a terminal you send it yourself with **Ctrl-D**. This zero is the single most important number in the next fifty slides.

`printf` is a buffered, formatting wrapper around `write`. It takes format strings and variable arguments, translating them into raw bytes inside a user-space buffer instead of immediately sending them to the kernel.

Unlike `write`, it returns an int representing the number of characters formatted, not bytes written. Because it **defers** the actual write until a flush is triggered, a program crash can lose buffered `printf` output that `write` would have already **committed** to the OS.

`printf` talks to a buffer; `write` talks to the kernel.

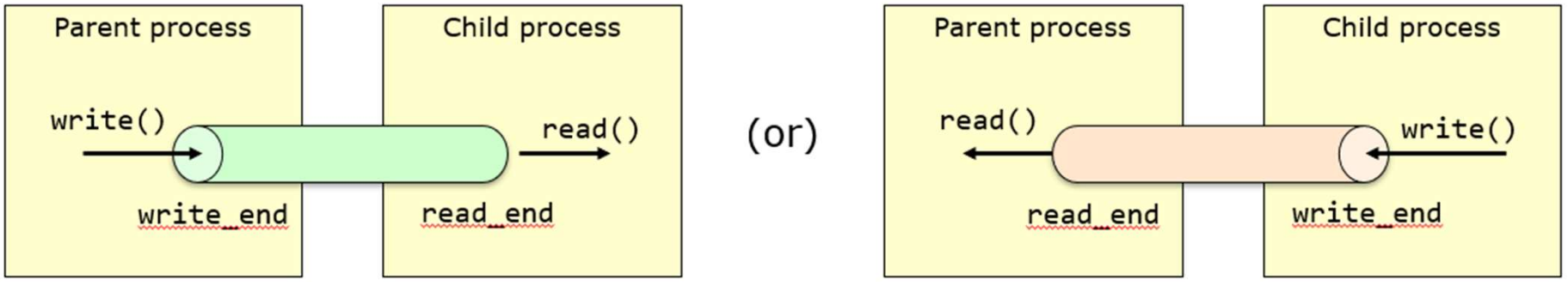
What is a PIPE?

A pipe is a fundamental mechanism for **Inter-Process Communication (IPC)** in Unix-like operating systems.

Imagine it's a one-way water pipe connecting two points.



- One process **writes** data to one end. ✍
- Another process **reads** that data from the other end. 📖
- It's a **unidirectional** (one-way) communication channel.
- The data flows in a First-In, First-Out (FIFO) manner.

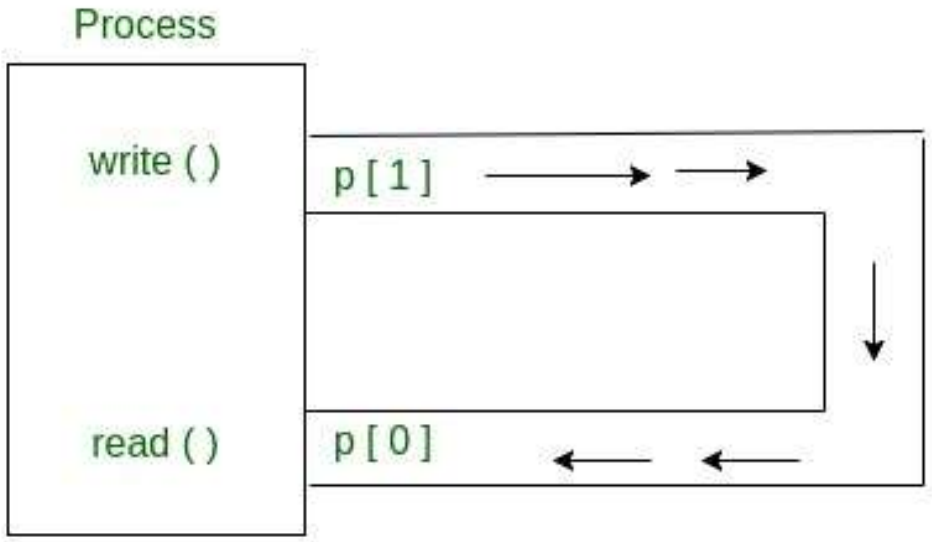


A pipe can be created using the system call **`pipe`**

```
#include <unistd.h>
int pipe (int fileDescr[2]);
```

`pipe(int fd[2])`: One Call, Two Ends

```c
int fd[2];                          /* fd[0] = read end, fd[1] = write end */
if (pipe(fd) == -1) {               /* one syscall, two file descriptors   */
    perror("pipe");
    return EXIT_FAILURE;
}
```



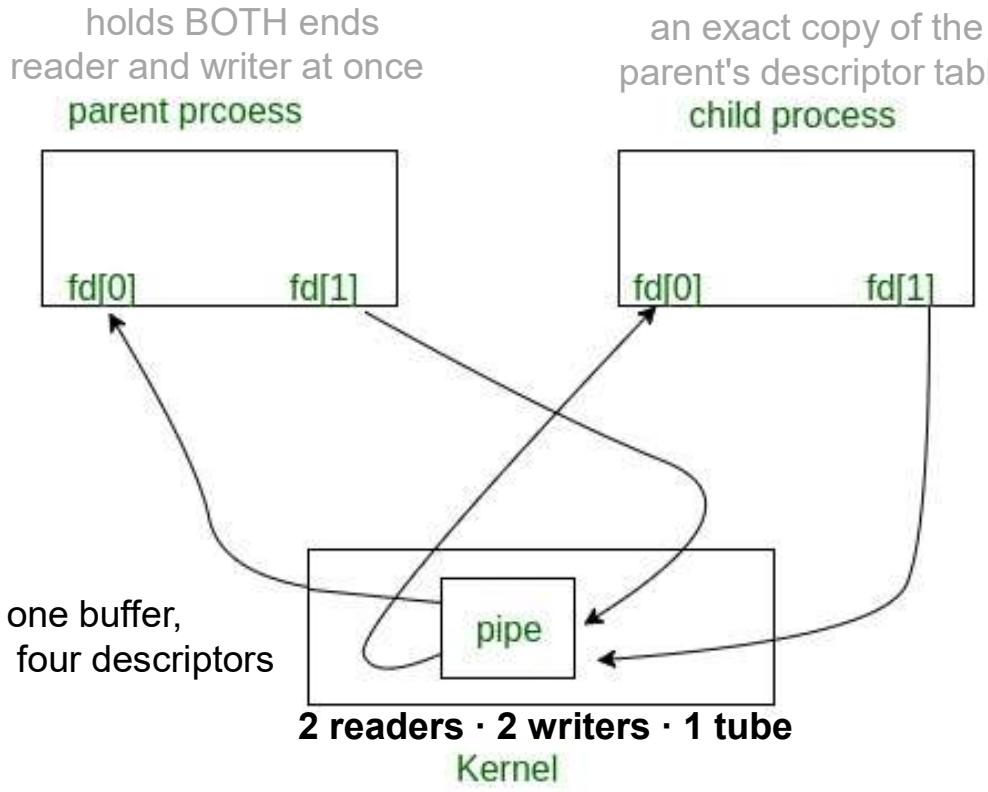
The mnemonic that saves you an hour

0 is stdin-like, so it is where you read.
1 is stdout-like, so it is where you write.
A **pipe** is a nameless, fixed-size buffer inside the kernel with two descriptors on it:
fd[0] the read end, **fd[1] the write end**.

pipe capacity — the buffer's size: **65536 bytes** (64 KiB) on Linux since kernel 2.6.11; before that it was one page.

A **pipe** is like a **file** because the kernel represents **each end with a file descriptor**, so processes can use read(), write(), and close() on it just as they would on an open file.

Now Add `fork()`, and Count the Ends



And that is the danger

Four open ends means the kernel sees **two writers and two readers**, and it cannot tell which of them you meant. In particular it will not declare the stream finished while *any* write end is still open — including one held by the process that is trying to read. Nothing crashes. Nothing logs. The program simply stops (deadlock), forever.

The standard [Correct] flow of operation is:

- A process creates a pipe; then it performs a fork
- The child processes inherit the file descriptors
- One of the two processes (e.g. father) writes in the pipe while the other (e.g. child) reads from the pipe
- The unused descriptor can/**MUST** be closed

The Sacred Rule of Closing

every process closes every pipe end it will not use, immediately after `fork`. Not tidiness — correctness

```
if (pid == 0) {                                  /* child: the reader */
    if (close(fd[1]) == -1) {                    /* ⟵ close unused WRITE end */
        perror("close"); _exit(EXIT_FAILURE);
    }
…  the child's read loop, lines 23-31  …
/* parent: the writer */
if (close(fd[0]) == -1) { perror("close"); return EXIT_FAILURE; }   /* ⟵ close unused READ end */
const char *msg = "message from parent to child\n";
if (write(fd[1], msg, strlen(msg)) == -1) { perror("write"); return EXIT_FAILURE; }
if (close(fd[1]) == -1) { perror("close"); return EXIT_FAILURE; }   /* triggers child EOF */
```

The two crime-scene tags

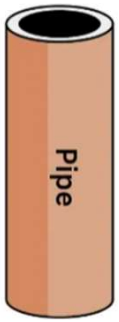
The child closes `fd[1]` : it will never write. The parent closes `fd[0]` : it will never read. And the **last** close — the parent's own `fd[1]` — is what finally tells the child the conversation is over.

Example

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>a
#include <string.h>
int main(){
        int p[2];          char buf[80]="Hello";          pid_t pid;
char buf2[80];
         pipe(p);
        if ((pid=fork()) == 0)  {
                                        /* in child process */
                close(p[0]);            /*close unused read */
                write(p[1],buf,strlen(buf)+1);
                        /*write it out, including
                        /*null byte */
                exit(0);
        }
                                        /*in parent process*/
        close(p[1]);            /*close unused write side of pipe */
        read(p[0],buf2,sizeof(buf2));      /*read the pipe*/
        printf("Child process said: %s\n", buf2);

        exit(0);
}
```

`write()` **blocks/sleeps** until the reader drains some data when pipe Is full

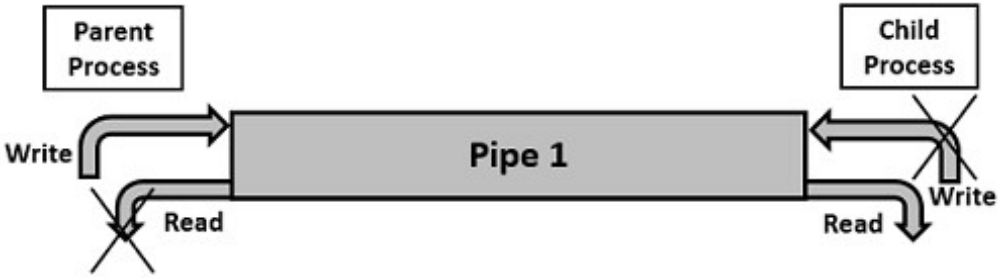


EOF (end of file) on a pipe — read() returns **0**, meaning "no more data will ever arrive". It is not an error and not a byte; it is a verdict.

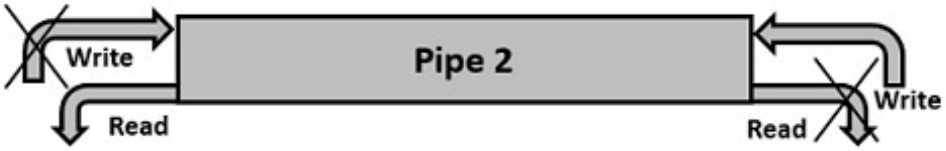
When a process tries to write to a pipe whose **read end is already closed**, the kernel sends SIGPIPE to the writer.

TWO DIRECTIONS = TWO PIPES

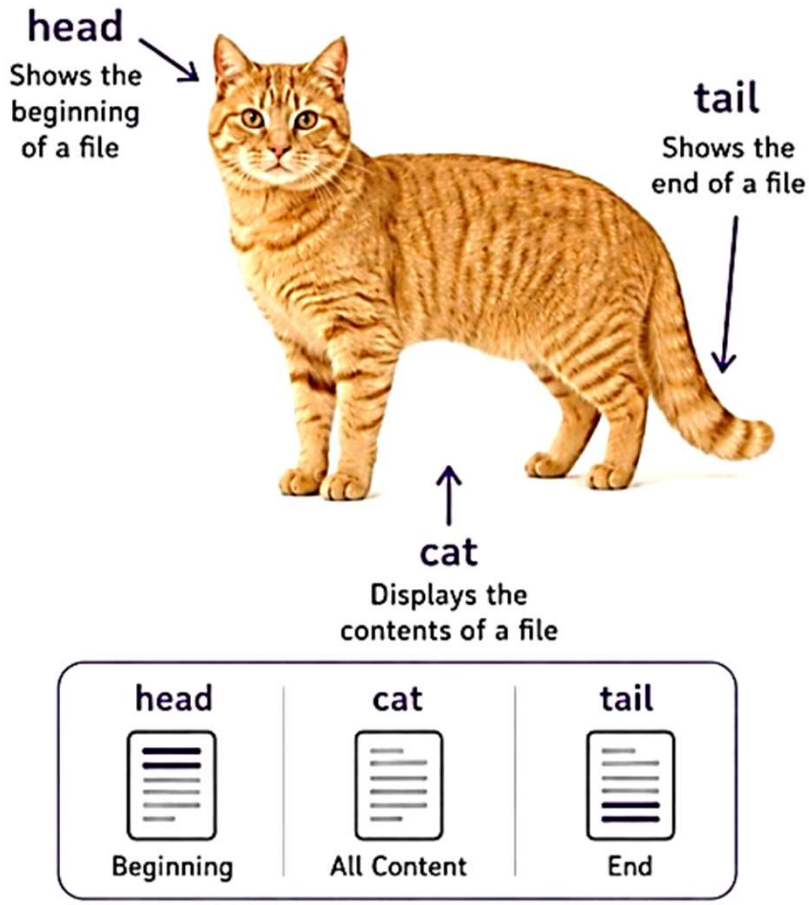
```
// If this is the child process
  else if (p == 0) {
    // Close the write end of the first pipe
    close(pipe1[1]);
    // Close the read end of the second pipe
    close(pipe2[0]);
    // Write the message "gfg" to the second pipe
    write(pipe2[1], m1, sizeof(m1));
    // Read a message from the first pipe
    read(pipe1[0], m_store, sizeof(m_store));
    // Print the received message
    printf("Child reads %s \n", m_store);
    // Close the read end of the first pipe
    close(pipe1[0]);
    // Close the write end of the second pipe
    close(pipe2[1]);
  }
  // If this is the parent process
  else {
    // Close the write end of the second pipe
    close(pipe2[1]);
    // Close the read end of the first pipe
    close(pipe1[0]);
    // Write the message "GKG" to the first pipe
    write(pipe1[1], m2, sizeof(m2));
    // Read a message from the second pipe
    read(pipe2[0], m_store, sizeof(m_store));
    // Print the received message
    printf("Parent reads %s \n", m_store);
    // Close the read end of the second pipe
    close(pipe2[0]);
    // Close the write end of the first pipe
    close(pipe1[1]);
  }
```

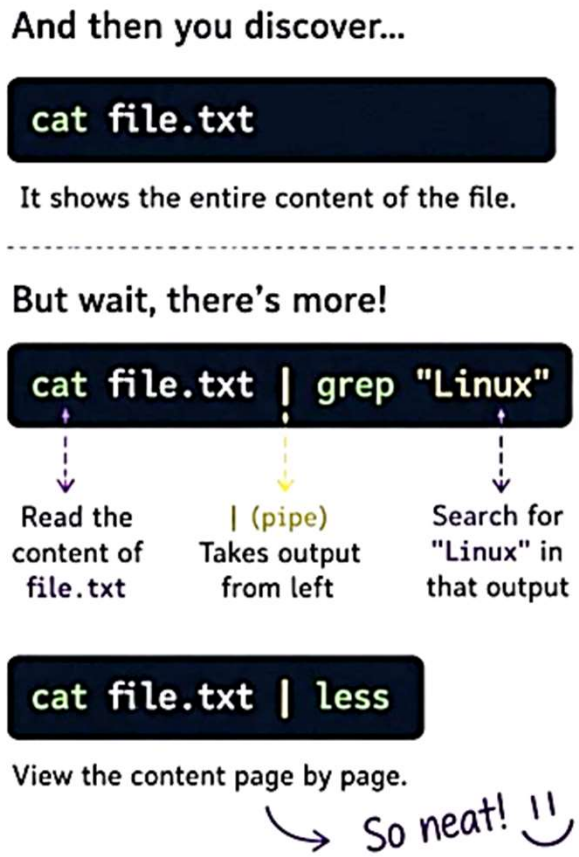


one pipe is half-duplex: data flows one way only



Shell Support Pipes



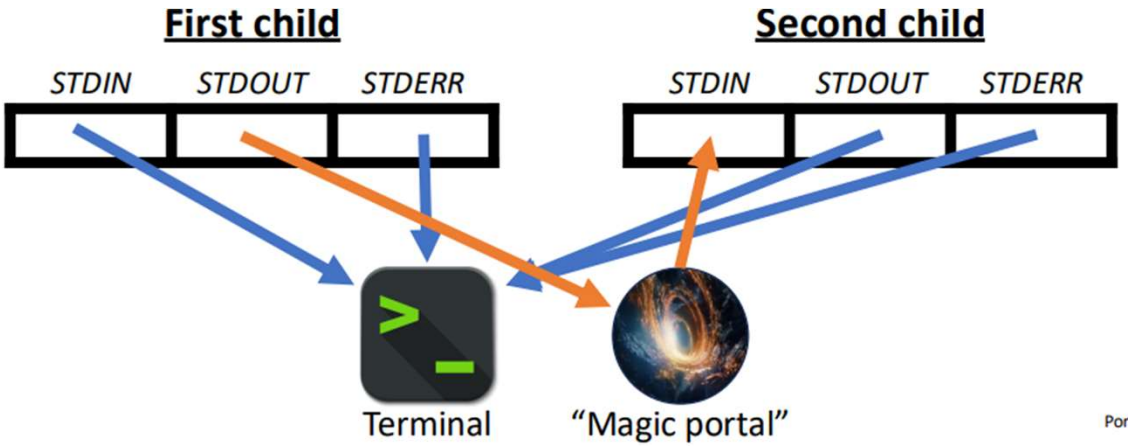


The shell creates a pipe, forks both processes, and somehow wires the pipe into their standard input and output.

If `cat` writes to `fds[1]`, how does `grep` know to read from `fds[0]` instead of the terminal? `grep` doesn't know about your pipe's file descriptors!

Rewiring File Descriptors

The previous parent-child pipe program requires manually reading and writing from new file descriptors created for the pipe.



Portal image unmodified, CC credit

Ultimate goal: invisibly connect pipe to STDOUT/STDIN without the programs knowing. We want anything the ”writer process” prints to the terminal to be automatically written to the pipe. And want whatever the “reader process” reads from its STDIN to actually be read from the pipe.

```c
// Child process
if (fork() == 0) {
    close(p[0]);
    dup2(p[1], 1); // stdout is now the pipe
    printf("Hello from child\n"); // goes into the pipe
    exit(0);
}
```

Redirecting Process I/O

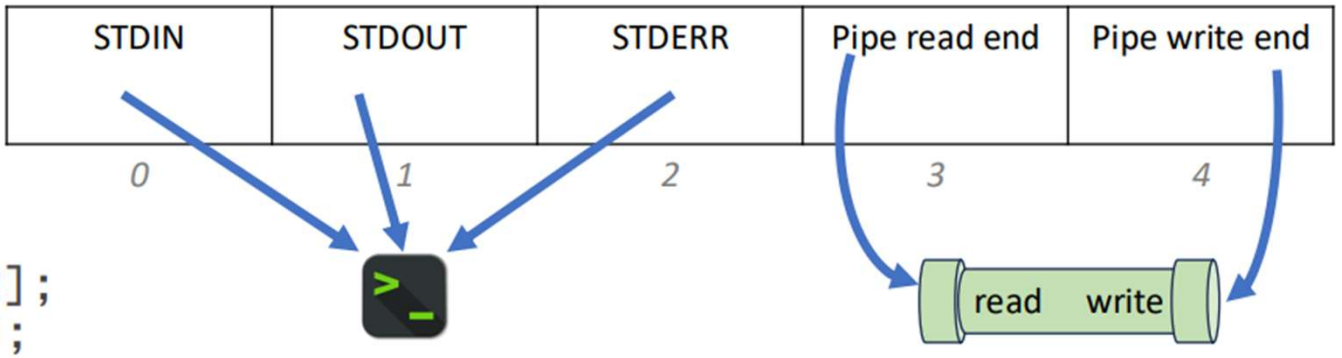
`dup2` copies over file descriptor information from one file descriptor number (`srcfd`) to another (`dstfd`).

```
int dup2(int srcfd, int dstfd);
```

e.g. `dup2(3, 1);`

// duplicates from 3 -> 1; FD 1 now also refers to what 3 is referring to.

- Both will now refer to the same thing (e.g. if you read from one, it will advance the position of the other).
- If `dstfd` is already open, it is closed before being used.




```
int fds[2];
pipe(fds);

printf("This prints to the terminal\n");

dup2(fds[1], STDOUT_FILENO); // equivalent here to dup2(4, 1)
printf("This is sent to the pipe!\n");
```

Redirecting Process I/O

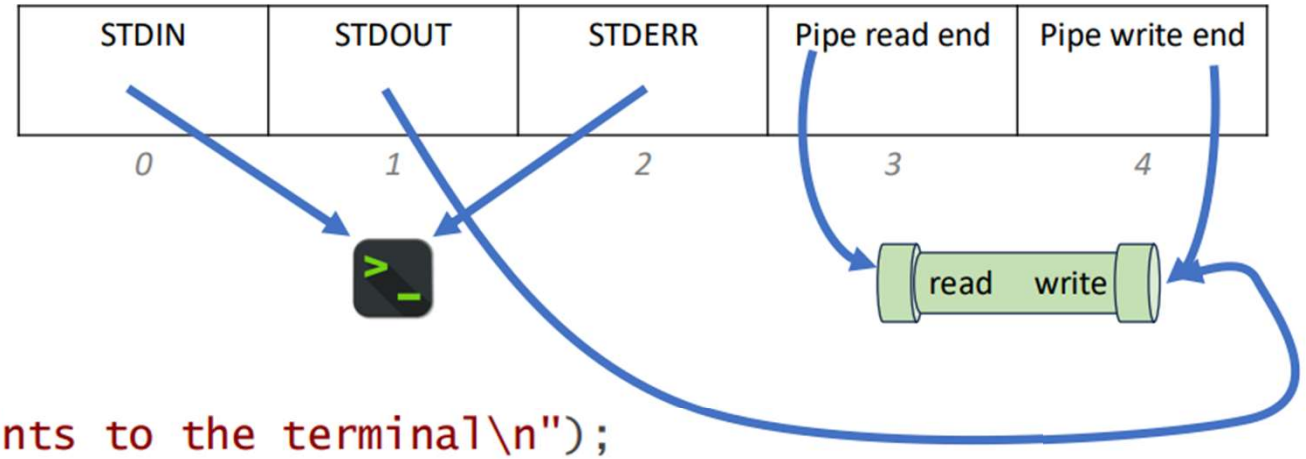
`dup2` copies over file descriptor information from one file descriptor number (`srcfd`) to another (`dstfd`).

```
int dup2(int srcfd, int dstfd);
```

e.g. `dup2(3, 1);`

// duplicates from 3 -> 1; FD 1 now also refers to what 3 is referring to.

- Both will now refer to the same thing (e.g. if you read from one, it will advance the position of the other).
- If `dstfd` is already open, it is closed before being used.



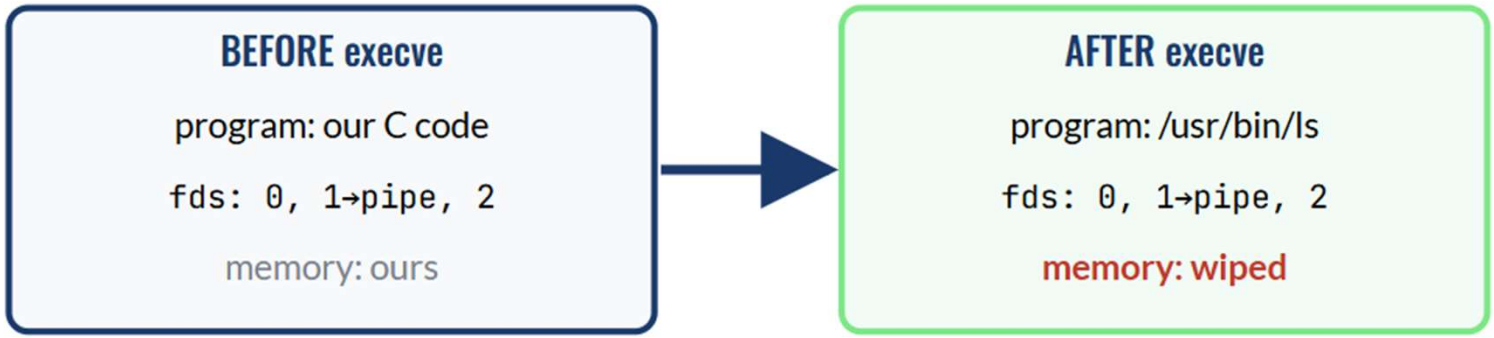

```c
int fds[2];
pipe(fds);

printf("This prints to the terminal\n");

dup2(fds[1], STDOUT_FILENO); // equivalent here to dup2(4, 1)
printf("This is sent to the pipe!\n");
```

exec Keeps the Descriptors — That Is the Whole Trick

`execve` throws away the program image — code, heap, stack, everything — and keeps the descriptor table



Descriptors are inherited by default; anything you do not want the next program to hold, you must mark or close.

Why you care, in the homework and the project

A descriptor leaked into a long-lived child is a write end you cannot close — and therefore an EOF that never arrives, in a process you were not even debugging.

```
// In the child for the `ls`:
dup2(p[1], 1);
// 1. Rewire stdout to the pipe
close(p[0]);
close(p[1]);
// 2. Close unused ends
execlp("ls", "ls", NULL);
// 3. Replace memory with `ls`
```

We use **dup2** to secretly rewire the child's *stdout* to the pipe and close the leftover ends to ensure proper **EOF**, then **exec** replaces the child with **ls**. Because exec preserves the file descriptor table, **ls** blindly writes to stdout and unknowingly (we abstract the pipe away) sends its output straight into the pipe instead of the terminal.



Questions?